

# D&D 3.5e Prototype — Codebase Analysis Report

**Date:** May 27, 2026  
**Scope:** All 623 C# files (277,474 lines) under `Assets/Scripts/`  
**Purpose:** Identify opportunities for improving efficiency, reusability, and systemization

## Executive Summary

The prototype is a substantial codebase (~277K lines) with a mature feature set covering character creation, combat, spellcasting, equipment, AI, and encounter systems. The architecture makes heavy use of C# partial classes to split large systems across files, which aids development velocity but has created several systemic issues:

| Category      | Findings | High Impact | Medium | Low |
|---------------|----------|-------------|--------|-----|
| Efficiency    | 14       | 4           | 6      | 4   |
| Reusability   | 12       | 5           | 4      | 3   |
| Systemization | 10       | 3           | 5      | 2   |
| Total         | 36       | 12          | 15     | 9   |

### Top 3 Impact Areas:

- GameManager God Object** (48K lines, 45 partial files) — needs service extraction
- NPCDatabase Boilerplate** (15K lines, 31 files) — needs data-driven approach
- Spell Implementation Duplication** (10K lines) — needs spell pipeline abstraction

## Codebase Overview

### Size Distribution

| Directory  | Lines  | Files | Description                                                 |
|------------|--------|-------|-------------------------------------------------------------|
| Core/      | 48,444 | 54    | GameManager (45 files), SceneBootstrap, events              |
| Character/ | 45,993 | 65    | NPCDatabase (31 files), CharacterStats, CharacterController |
| UI/        | 37,491 | 46    | CombatUI, CharacterSheet, CreationUI, Panels                |
| Magic/     | 34,207 | 105   | Spells (24 DB files), StatusEffects (28), AreaEffects (31)  |
| Tests/     | 30,090 | 92    | Comprehensive test suite                                    |
| Equipment/ | 18,637 | 61    | Factories, Behaviors (28 files), materials                  |
| Inventory/ | 9,560  | 13    | ItemData, ItemDatabase, PotionFactory                       |
| Other      | 53,052 | 187   | Services, Combat, AI, Classes, Grid, etc.                   |

## Largest Files (Hotspots)

| File                        | Lines  | Concern                             |
|-----------------------------|--------|-------------------------------------|
| CharacterController.cs      | 12,126 | Multiple responsibilities           |
| GameManager.cs              | 11,110 | God object root                     |
| GameManager.SpellCasting.cs | 9,021  | Spell dispatch + resolution         |
| CharacterStats.cs           | 6,088  | Stats + conditions + saves + skills |
| CombatUI.cs                 | 4,661  | UI monolith                         |
| PreCombatInventoryUI.cs     | 4,637  | UI monolith                         |
| SpellcastingComponent.cs    | 3,106  | Spell slot management               |

## Category 1: Efficiency

### E1. Class Identity Lookups via String Iteration — High Impact

**Location:** CharacterStats.cs:427-516

GetClassLevel() iterates a List<ClassLevelEntry> with string.Equals() on every call. This feeds 13+ boolean properties (IsRogue, IsWizard, IsCleric, IsBard, IsDruid, IsMonk, IsPaladin, IsRanger, IsSorcerer, IsBarbarian, IsFighter, etc.) that are accessed frequently during combat resolution.

```
// CURRENT (O(n) per call, called 13+ times per stat check)
public int GetClassLevel(string className)
{
    for (int i = 0; i < ClassLevels.Count; i++)
        if (string.Equals(ClassLevels[i].ClassName, className, StringComparison.OrdinalIgnoreCase))
            return ClassLevels[i].Level;
    return 0;
}
public bool IsRogue => HasClass("Rogue"); // triggers full iteration
```

**Recommendation:** Cache class levels in a Dictionary<string, int> on initialization and invalidate on level-up only. Or use a [Flags] enum CharacterClassFlags with a single bitwise check:

```
// PROPOSED
private Dictionary<string, int> _classLevelCache;
private CharacterClassFlags _classFlags;

[Flags] enum CharacterClassFlags {
    None=0, Fighter=1, Rogue=2, Wizard=4, Cleric=8, Barbarian=16,
    Bard=32, Druid=64, Monk=128, Paladin=256, Ranger=512, Sorcerer=1024
}

public bool IsRogue => (_classFlags & CharacterClassFlags.Rogue) != 0; // 0(1)
```

**Impact:** Every attack, AC check, save calculation, and feat check calls these properties.

## E2. ActiveConditions Linear Search in Hot Path — High Impact

**Location:** CharacterStats.cs:982-1006, 1335-1392

Condition checks ( CountConditionStacks , HasNormalizedCondition , AddCondition , RemoveCondition ) use List.FirstOrDefault() , FindIndex() , and iteration — all O(n) per call. These are invoked multiple times per combat action.

```
// CURRENT — O(n) on every condition check
private bool HasNormalizedCondition(CombatConditionType type)
    => ActiveConditions.FirstOrDefault(c => NormalizeCondition(c.ConditionType) == type).ConditionType != default;
```

**Recommendation:** Maintain a parallel Dictionary<CombatConditionType, List<StatusEffect>> or HashSet<CombatConditionType> for O(1) lookups, updated when conditions are added/removed.

## E3. GetComponent Calls Not Cached — Medium Impact

**Location:** GameManager\*.cs (290 calls), CharacterController.cs (42 calls)

Unity GetComponent<T>() is an expensive reflection-based operation. The codebase calls it 330+ times across GameManager and CharacterController, often repeatedly on the same object within the same method.

```
// CURRENT — GameManager.cs:934-987 (same character, 3 GetComponent calls)
StatusEffectManager statusMgr = GetComponent<StatusEffectManager>();
SpellcastingComponent spellComp = GetComponent<SpellcastingComponent>();
Inventory inventory = GetComponent<InventoryComponent>()?.CharacterInventory;
```

**Recommendation:** Cache component references in CharacterController.Awake() or use a ComponentCache helper:

```
// On CharacterController
public StatusEffectManager StatusEffects { get; private set; }
public SpellcastingComponent Spellcasting { get; private set; }
public InventoryComponent InventoryComp { get; private set; }

void Awake() {
    StatusEffects = GetComponent<StatusEffectManager>();
    Spellcasting = GetComponent<SpellcastingComponent>();
    InventoryComp = GetComponent<InventoryComponent>();
}
```

## E4. String Allocations on Property Access — 🟡 Medium Impact

**Location:** CharacterStats.cs:244, 450-453

Properties like `DomainsDisplay` and `ClassSummary` allocate new strings via `string.Join()` on every access, including during UI refresh cycles.

```
// CURRENT – allocates on every read
public string DomainsDisplay => ChosenDomains.Count > 0
    ? string.Join(", ", ChosenDomains) : "None";
public string ClassSummary => string.Join(" / ",
    ClassLevels.Select(cl => $"{cl.ClassName} {cl.Level}"));
```

**Recommendation:** Use dirty-flag caching — regenerate only when underlying data changes:

```
private string _classSummaryCache;
private bool _classSummaryDirty = true;
public string ClassSummary {
    get {
        if (_classSummaryDirty) {
            _classSummaryCache = string.Join(" / ", ClassLevels.Select(cl => $"{cl.Cla
ssName} {cl.Level}"));
            _classSummaryDirty = false;
        }
        return _classSummaryCache;
    }
}
```

## E5. NPCDatabase Init Registers All Creatures Eagerly — 🟡 Medium Impact

**Location:** NPCDatabase.cs:Init() — calls 31 `RegisterCreatures_X()` methods

All ~200+ creature definitions are registered at startup regardless of which encounter is loaded. Each creates full `NPCDefinition` objects with `List<>` allocations for attacks, feats, tags, equipment, etc.

**Recommendation:** Consider lazy registration — register only creature IDs and metadata at startup, defer full definition construction to first `Get()` call. Or split into “tier” files: SNA-summons-only tier loads first, rare/boss creatures load on demand.

## E6. Spell Dispatch via String-Based Switch — 🟡 Medium Impact

**Location:** `GameManager.SpellCasting.cs`

Spell resolution routes through string-matching patterns across 20+ spell implementation files. Each file's methods are invoked through `if/else` or `switch` chains matching `spell.SpellId`.

**Recommendation:** Register spell handlers in a `Dictionary<string, Action<SpellContext>>` during initialization:

```
private static Dictionary<string, Action<SpellCastContext>> _spellHandlers = new();
static void RegisterSpellHandlers() {
    _spellHandlers[SpellNames.MAGIC_MISSILE] = ResolveMagicMissile;
    _spellHandlers[SpellNames.FIREBALL] = ResolveFireball;
    // ...
}
void ResolveSpell(SpellCastContext ctx) {
    if (_spellHandlers.TryGetValue(ctx.Spell.SpellId, out var handler))
        handler(ctx);
}
```

## E7. List Allocations in Combat Hot Path — 🟢 Low Impact

**Locations:** Various places in `SupportActions.cs` and `CharacterController.cs`

Temporary `List<T>` allocations occur during target finding, AoE resolution, and feat enumeration. These create GC pressure during combat.

**Recommendation:** Use `ListPool<T>` or `stackalloc` where applicable. For small counts, use `Span<T>` or fixed-size arrays.

## E8. Color Construction in NPCDatabase — 🟢 Low Impact

**Location:** All `NPCDatabase_*.cs` files (570+ `new Color()` calls)

Every creature definition constructs 3 `Color` structs inline. While `Colors` are value types (no GC), identical colors are reconstructed redundantly.

**Recommendation:** Define a `CreatureColorPalette` static class with named constants:

```
public static class CreatureColorPalette {
    // Undead
    public static readonly Color UndeadSprite = new Color(0.5f, 0.5f, 0.65f, 1f);
    public static readonly Color UndeadPanel = new Color(0.15f, 0.12f, 0.2f, 0.85f);
    public static readonly Color UndeadName = new Color(0.7f, 0.65f, 0.85f);
    // Animal
    public static readonly Color AnimalSprite = new Color(0.7f, 0.55f, 0.35f, 1f);
    // ...
}
```

## Category 2: Reusability

### R1. NPCDefinition Boilerplate — 409 Redundant Empty List Assignments — High Impact

**Location:** All 31 `NPCDatabase_*.cs` files (14,843 lines total)

Every creature definition repeats:

```
EquipmentIds = new List<EquipmentSlotPair>(), // 191 occurrences
BackpackItemIds = new List<string>(), // 218 occurrences
```

These are already default-initialized in `NPCDefinition` :

```
// NPCDatabase.cs - NPCDefinition class already has defaults!
public List<EquipmentSlotPair> EquipmentIds = new List<EquipmentSlotPair>();
public List<string> BackpackItemIds = new List<string>();
```

**Recommendation:** Remove all 409 redundant assignments. Fields already have defaults. This alone removes ~800 lines of boilerplate. Similarly, check `CreatureTags` , `Feats` , `SpecialAbilities` , etc. — any that just reassign the default can be removed.

**Estimated savings:** 800-1,200 lines across `NPCDatabase` files.

### R2. Saving Throw Resolution Pattern Duplicated 12+ Times — High Impact

**Location:** `GameManager_Spells_Phase1.cs:82-721` and across all `GameManager_Spells_*.cs` files

The identical 6-line save resolution block appears 12+ times just in `Phase1.cs` :

```
// This exact pattern appears 12+ times with only the save type varying
int willRoll = UnityEngine.Random.Range(1, 21);
int willMod = target.Stats.WillSave;
int willTotal = willRoll + willMod;
bool saved = willTotal >= saveDc;
sb.AppendLine($" Will: d20({willRoll}) + {willMod} = {willTotal} vs DC {saveDc} → {(saved ? "SAVED (negated)" : "FAILED")}");
if (saved) { sb.AppendLine(); continue; }
```

With 67 DC/save references across spell files, this is the most duplicated combat pattern.

**Recommendation:** Extract a `SpellSaveResolver` utility:

```

public static class SpellSaveResolver {
    public struct SaveResult {
        public bool Saved;
        public int Roll, Modifier, Total, DC;
        public string SaveType;
    }

    public static SaveResult MakeSave(CharacterController target, SavingThrowType type
, int dc) {
        int roll = UnityEngine.Random.Range(1, 21);
        int mod = type switch {
            SavingThrowType.Will => target.Stats.WillSave,
            SavingThrowType.Reflex => target.Stats.ReflexSave,
            SavingThrowType.Fortitude => target.Stats.FortitudeSave,
            _ => 0
        };
        return new SaveResult {
            Roll = roll, Modifier = mod, Total = roll + mod,
            DC = dc, Saved = (roll + mod) >= dc,
            SaveType = type.ToString()
        };
    }

    public static void AppendToLog(StringBuilder sb, SaveResult result, string onFail-
Text = "FAILED") {
        sb.AppendLine($" {result.SaveType}: d20({result.Roll}) + {result.Modifier} =
" +
            $"{result.Total} vs DC {result.DC} → {(result.Saved ? "SAVED" : onFail-
Text)}");
    }
}

```

### R3. Spell Implementation Lacks Pipeline Abstraction — High Impact

**Location:** 22 `GameManager_Spells_*.cs` files (9,763 lines total)

Each spell implementation manually handles the same sequence:

1. Validate targets/range
2. Check Spell Resistance (55 SR references across files)
3. Roll saving throw
4. Apply damage or effect
5. Log result to combat log

Some helper methods exist in `GameManager_Spells_Shared.cs` ( `TryResolveScaledAoEDamageSpell` , `ResolveAlignmentBurstSpell` ) but they're one-offs, not a systematic pipeline.

**Recommendation:** Create a `SpellPipeline` that standardizes the resolution flow:



```

public class SpellResolutionContext {
    public CharacterController Caster;
    public SpellData Spell;
    public List<CharacterController> Targets;
    public int CasterLevel, SaveDC;
    public StringBuilder Log;
}

public abstract class SpellResolver {
    public virtual bool CheckSpellResistance(SpellResolutionContext ctx, CharacterCon-
troller target) { ... }
    public virtual SpellSaveResolver.SaveResult? TrySave(SpellResolutionContext ctx, C
haracterController target) { ... }
    public abstract void ApplyEffect(SpellResolutionContext ctx, CharacterController t
arget, bool saved);
}

// Example: specific spell just overrides ApplyEffect
public class FireballResolver : SpellResolver {
    public override void ApplyEffect(SpellResolutionContext ctx, CharacterController t
arget, bool saved) {
        int dice = Math.Min(ctx.CasterLevel, 10);
        int damage = DiceRoller.Roll(dice, 6);
        if (saved) damage /= 2;
        target.TakeDamage(damage, DamageType.Fire);
    }
}

```

This would reduce 9,763 lines of spell code by an estimated 30-40%.

## R4. No Shared Factory Base for Item Creation — High Impact

**Location:** 8 Factory files: `WondrousItemFactory.cs` (2,299 lines), `RingFactory.cs`, `RodFactory.cs`, `PotionFactory.cs`, `ScrollFactory.cs`, `WandFactory.cs`, `EnchantmentFactory.cs`, `ItemMaterialFactory.cs`

Each factory independently constructs `ItemData` objects with repeated field assignments. `WondrousItemFactory` has its own `CreateBaseWondrous()` helper, `RingFactory` constructs inline, etc.

**Recommendation:** Extract a shared `ItemBuilder` fluent API:

```

public class ItemBuilder {
    private readonly ItemData _item = new ItemData();

    public ItemBuilder(string id, string name) { _item.Id = id; _item.Name = name; }
    public ItemBuilder Type(ItemType type) { _item.Type = type; return this; }
    public ItemBuilder Slot(EquipSlot slot) { _item.Slot = slot; return this; }
    public ItemBuilder Price(int gp) { _item.BasePriceGp = gp; return this; }
    public ItemBuilder CasterLevel(int cl) { ... return this; }
    public ItemBuilder Icon(string icon, Color color) { ... return this; }
    public ItemData Build() => _item;
}

// Usage
var ring = new ItemBuilder("ring_protection_1", "Ring of Protection +1")
    .Type(ItemType.Ring).Slot(EquipSlot.EitherRing)
    .Price(2000).CasterLevel(3)
    .Icon("🔮", Color.blue)
    .Build();

```

## R5. Creature Definition Templates Not Fully Exploited — High Impact

**Location:** NPCDatabase\_M.cs:90-154 (MephitisBase), all NPCDatabase files

The `MephitisBase()` factory method shows this works — it creates 10 mephitis variants with shared stats. But this is the **only** factory pattern in 14,843 lines of creature definitions.

Similar creature families that should use the same pattern:

- **Monstrous Scorpions** (Small/Medium/Large/Huge) — differ only in size + stats
- **Monstrous Spiders** (Small/Medium/Large/Huge) — same pattern
- **Vipers** (Small/Medium/Large/Huge) — same pattern
- **Dire Animals** — many share structure (Animal type, scent, aggressive melee)
- **Elementals** (Small/Medium/Large/Huge × 4 elements) — 16 variants from 1 template

**Recommendation:** Create factory methods for each creature family:

```

private static NPCDefinition MonstousScorpionBase(SizeCategory size, int hd, int str,
int dex, int con,
    int naturalArmor, int speed, int clawDice, int stingDice, int poisonDC) {
    return new NPCDefinition {
        Id = $"monstrous_scorpion_{size.ToString().ToLower()}",
        Name = $"Monstrous Scorpion, {size}",
        CreatureType = "Vermin", SizeCategory = size,
        // ... shared vermin traits
    };
}

```

**Estimated savings:** 2,000-3,000 lines by templating the 15+ creature families.

## R6. Equipment Behavior Classes Have No Shared Base — Medium Impact

**Location:** Assets/Scripts/Equipment/Behaviors/ — 28 files

Each behavior file ( `HolyAvengerBehavior` , `FrostBrandBehavior` , etc.) independently implements similar patterns for on-equip/unequip bonuses, on-hit effects, and activation. Some extend `SpecificItemBehavior` but there's no standardized interface for common operations.

**Recommendation:** Define `IEquipBehavior` with standard hooks:

```
public interface IEquipBehavior {
    void OnEquip(CharacterController wielder);
    void OnUnequip(CharacterController wielder);
    void OnHit(CharacterController wielder, CharacterController target, CombatResult result);
    void OnTurnStart(CharacterController wielder);
    bool CanActivate(CharacterController wielder);
    void Activate(CharacterController wielder);
}
```

## R7. Area Effect Subclasses Duplicate Lifecycle Logic — Medium Impact

**Location:** `Assets/Scripts/Magic/AreaEffects/` — 31 files, ~20 extending `PersistentAreaEffect`

The `PersistentAreaEffect` base class is well-designed but subclasses often re-implement similar patterns for:

- On-enter save throws
- Per-round damage ticks
- Movement penalties
- Concealment/vision blocking

**Recommendation:** Add composable behaviors to `PersistentAreaEffect` :

```
// In PersistentAreaEffect
public int OnEnterDamageDice, OnEnterDamageCount;
public DamageType OnEnterDamageType;
public SavingThrowType OnEnterSaveType;
public bool OnEnterSaveNegates, OnEnterSaveHalves;
public float MovementMultiplier = 1f;
public bool BlocksLineOfSight;
```

## R8. Status Effect Files Lack Common Base — Medium Impact

**Location:** `Assets/Scripts/Magic/StatusEffects/` — 28 files

Unlike AreaEffects which have `PersistentAreaEffect` , status effects have no shared base class (only `EmanationEffectData` for a subset). Each file manually handles duration tracking, stacking rules, and removal.

**Recommendation:** Create `BaseStatusEffect` with standard duration, stacking, and removal logic.

## R9. Identical Creature Stat Blocks Not Shared — Low Impact

**Location:** Multiple NPCDatabase files

At least 3 pairs of creatures have identical ability scores:

- Djinni + Noble Djinni (STR 18, DEX 17, CON 14, WIS 15, INT 14, CHA 15)
- Monstrous Centipede Medium + Monstrous Spider Small (STR 9, DEX 15, CON 10, etc.)

**Recommendation:** Use a base definition with `Clone()` and overlay:

```
var noble = Get("djinni").Clone();
noble.Id = "noble_djinni";
noble.Name = "Noble Djinni";
noble.ChallengeRating = "8";
Register(noble);
```

---

## Category 3: Systemization

### S1. GameManager God Object — 48K Lines Across 45 Partial Files — High Impact

**Location:** `Assets/Scripts/Core/GameManager*.cs` — 45 files, 48,444 lines

`GameManager` is the largest class in the codebase, responsible for:

- Combat flow and turn management
- Spell resolution (22 spell files)
- NPC turn AI
- Loot collection
- Wall/area spell effects (5 files)
- Test configurations
- Dungeon encounters
- Dispel/counterspell
- Mirror image
- Domain powers and spells

This violates Single Responsibility Principle severely. Changes to spell logic risk breaking combat flow; test configs share state with production code.

**Recommendation:** Extract focused service classes:

| Current (GameManager partial)                 | Proposed Service       |
|-----------------------------------------------|------------------------|
| GameManager.SpellCasting.cs +<br>_Spells_*.cs | SpellResolutionService |
| GameManager.CombatActions.cs                  | CombatActionService    |
| GameManager.NPCTurns.cs                       | NPCTurnService         |
| GameManager.LootCollection.cs                 | LootService            |
| GameManager_Wall*.cs + _FlamingSphere.cs      | AreaEffectService      |
| GameManager.TestConfigs.cs +<br>TestPanel.cs  | TestConfigService      |
| GameManager_Domain*.cs                        | DomainService          |

Start with the easiest: extract `TestConfigService` (1,875 lines) and `LootService` to prove the pattern.

## S2. SpellDatabase Organization — 295 Spells in 24 Alphabetical Files — High Impact

**Location:** `Assets/Scripts/Magic/Spells/Databases/SpellDatabase_*.cs`

Spells are organized purely alphabetically (like `NPCDatabase`). `SpellData` has 110+ fields with many spell-specific flags. Finding all spells of a given school, level, or class requires scanning all 24 files.

**Recommendation:** Consider:

1. **Data-driven:** Move spell definitions to JSON/YAML files loaded at runtime
2. **At minimum:** Add indexing after registration:

```
// Post-registration indexing
private static Dictionary<string, List<SpellData>> _bySchool;
private static Dictionary<int, List<SpellData>> _byLevel;
private static Dictionary<string, List<SpellData>> _byClass;

public static List<SpellData> GetSpellsBySchool(string school)
    => _bySchool.TryGetValue(school, out var list) ? list : new();
```

## S3. Data-Driven Approach for Creature Definitions — High Impact

**Location:** All `NPCDatabase_*.cs` files (14,843 lines)

Creature definitions are currently C# code compiled into the assembly. Every stat change requires recompilation. The `NPCDefinition` class has 76 fields — perfectly suitable for serialization.

**Recommendation:** Move creature definitions to JSON files, keep C# for the registration framework:

```
// creatures/satyr.json
{
  "id": "satyr",
  "name": "Satyr",
  "challengeRating": "2",
  "level": 5,
  "creatureType": "Fey",
  "hitDice": 5,
  "sizeCategory": "Medium",
  "str": 10, "dex": 13, "con": 12, "wis": 13, "int": 12, "cha": 13,
  "bab": 2,
  "naturalArmorBonus": 4,
  "baseSpeed": 8,
  "baseHitDieHP": 22,
  "damageReduction": { "amount": 5, "bypass": "ColdIron" },
  "naturalAttacks": [
    { "name": "Head butt", "damageDice": 6, "damageCount": 1, "count": 1,
    "isPrimary": true }
  ]
}
```

This would:

- Reduce compiled code by ~14K lines
- Allow modding/balancing without recompilation
- Enable tooling (creature editor, batch validators)
- Support runtime creature generation

## S4. No Centralized Dice Roller / RNG Utility — 🟡 Medium Impact

**Location:** Throughout codebase

`UnityEngine.Random.Range(1, 21)` appears inline in 100+ locations. No centralized dice rolling utility exists for:

- Consistent logging of all rolls
- Deterministic replay / testing
- Seeded RNG for reproducibility

**Recommendation:**

```
public static class DiceRoller {
  public static int D20() => UnityEngine.Random.Range(1, 21);
  public static int Roll(int count, int sides) {
    int total = 0;
    for (int i = 0; i < count; i++)
      total += UnityEngine.Random.Range(1, sides + 1);
    return total;
  }
  public static int RollWithLog(int count, int sides, StringBuilder log = null) { ..
  . }
}
```

## S5. Missing Enum for Class Names — 🟡 Medium Impact

**Location:** `CharacterStats.cs` , `ClassRegistry.cs` , `SpellData.ClassList` , all class files

Class names are passed as strings throughout: `"Fighter"` , `"Rogue"` , `"Wizard"` , etc. This causes:

- Typo risk (no compile-time checking)
- Case-sensitivity issues (needs `StringComparison.OrdinalIgnoreCase` )
- String allocation overhead

**Recommendation:** Define `CharacterClassName` enum and migrate:

```
public enum CharacterClassName {
    Fighter, Rogue, Wizard, Cleric, Barbarian,
    Bard, Druid, Monk, Paladin, Ranger, Sorcerer,
    // NPC classes
    Warrior, Adept, Expert, Aristocrat, Commoner
}
```

---

## S6. Inconsistent Identifier Systems — 🟡 Medium Impact

**Location:** `Assets/Scripts/Identifiers/` — 14 files

The project has multiple ID systems:

- `SpellNames` — string constants ( `SpellNames.FIREBALL = "fireball"` )
- `ItemIDs` — string constants ( `ItemIDs.LONGSWORD = "longsword"` )
- `RingNames` , `RodNames` , `WondrousItemNames` — separate ID files
- `NPCDatabase` uses bare string IDs ( `"satyr"` , `"dire_wolf"` )

No unified ID validation or centralized ID registry exists.

**Recommendation:** Standardize on a single pattern. The `SpellNames` const-string approach is good; extend it to all domains with compile-time validation:

```
namespace DND35e.Identifiers {
    public static class CreatureIds {
        public const string SATYR = "satyr";
        public const string DIRE_WOLF = "dire_wolf";
        // ... auto-generated from NPCDatabase registration
    }
}
```

---

## S7. Test Infrastructure Not Standardized — 🟡 Medium Impact

**Location:** `Assets/Scripts/Tests/` — 92 files, 30K lines

Tests use a custom framework with `MockCharacterFactory` but lack:

- Shared test fixtures for common setups
- Consistent assertion patterns
- Integration test vs unit test separation

**Recommendation:** Standardize test helpers:

```
public static class TestFixtures {
    public static CharacterController CreateFighter(int level = 3) { ... }
    public static CharacterController CreateWizard(int level = 5) { ... }
    public static void AssertDamageInRange(int actual, int min, int max) { ... }
    public static void AssertSaveResult(bool expected, SavingThrowType type, int dc)
    { ... }
}
```

## S8. Event System Underutilized — Medium Impact

**Location:** Assets/Scripts/Core/GameEventSystem.cs — 116 lines

A `GameEventSystem` with `IGameEvent` interface exists but is minimally used. Most communication between systems goes through direct method calls on `GameManager` or `GetComponent` chains.

**Recommendation:** Use events for decoupled communication:

```
// Events
public class SpellCastEvent : IGameEvent { public SpellData Spell; public Character-
Controller Caster; }
public class DamageTakenEvent : IGameEvent { public int Amount; public DamageType Type
; }
public class ConditionAppliedEvent : IGameEvent { public CombatConditionType Type; }

// Systems subscribe
GameEventSystem.Subscribe<DamageTakenEvent>(OnDamageTaken);
```

## S9. Hard-Coded Game Constants Scattered — Low Impact

**Location:** Throughout codebase

D&D constants are embedded inline:

- `Range(1, 21)` for d20 rolls (should use `DiceRoller.D20()`)
- Size modifiers, BAB progressions, save progressions as inline numbers
- Spell range formulas scattered across spell implementations

A `GameConstants.cs` file exists in `Identifiers/` but is underutilized.

**Recommendation:** Expand `GameConstants.cs` :



```

public static class GameConstants {
    public const int D20_MIN = 1, D20_MAX = 20;
    public const int FULL_ATTACK_BAB_THRESHOLD = 6;
    public const float CLOSE_RANGE_BASE_FT = 25f;
    public const float CLOSE_RANGE_PER_2_CL_FT = 5f;
    // Size modifiers
    public static readonly Dictionary<SizeCategory, int> SizeACModifier = new() {
        { SizeCategory.Fine, 8 }, { SizeCategory.Diminutive, 4 },
        { SizeCategory.Tiny, 2 }, { SizeCategory.Small, 1 },
        { SizeCategory.Medium, 0 }, { SizeCategory.Large, -1 },
        { SizeCategory.Huge, -2 }, { SizeCategory.Gargantuan, -4 },
        { SizeCategory.Colossal, -8 }
    };
}

```

## S10. No Formal Architecture Documentation — Low Impact

**Location:** Project-wide

No architecture diagram or system dependency map exists. The `docs/` folder contains only the SNA implementation plan. For a 277K-line codebase, this makes onboarding difficult.

**Recommendation:** Create `docs/architecture.md` documenting:

- System dependency graph (which services depend on what)
- Data flow: Character creation → Combat → Resolution → UI
- File organization conventions
- Adding new spells/creatures/items guide

## Priority Roadmap

### Phase 1: Quick Wins (1-2 days, high value/effort ratio)

1. **[R1]** Remove 409 redundant empty list assignments (~800 lines saved)
2. **[E1]** Cache class levels in Dictionary (performance win across entire codebase)
3. **[E3]** Cache GetComponent calls in CharacterController.Awake()
4. **[E4]** Add dirty-flag caching for string properties

### Phase 2: Structural Improvements (1-2 weeks)

1. **[R2]** Extract `SpellSaveResolver` utility (removes 12+ duplicated blocks)
2. **[S4]** Create `DiceRoller` utility class
3. **[R5]** Add creature family factory methods (Monstrous Scorpions, Vipers, etc.)
4. **[S1]** Extract `TestConfigService` from GameManager (easiest god-object slice)

### Phase 3: Architecture Evolution (2-4 weeks)

1. **[R3]** Design `SpellPipeline` abstraction for spell resolution
2. **[S1]** Continue GameManager decomposition (`SpellResolutionService`, `LootService`)
3. **[R4]** Create shared `ItemBuilder` for factory consolidation
4. **[S5]** Introduce `CharacterClassName` enum

Phase 4: Strategic Refactoring (1-2 months)

- 1. [S3] Migrate creature definitions to JSON (data-driven NPCDatabase)
- 2. [S2] Add spell database indexing by school/level/class
- 3. [S8] Expand event system usage for decoupled communication
- 4. [R7] Add composable behaviors to PersistentAreaEffect

Metrics Summary

| Metric                         | Current    | After Phase 1 | After All Phases |
|--------------------------------|------------|---------------|------------------|
| Total lines                    | 277,474    | ~275,000      | ~240,000         |
| NPCDatabase lines              | 14,843     | ~14,000       | ~2,000 (JSON)    |
| Spell impl lines               | 9,763      | ~9,500        | ~6,000           |
| GetComponent calls             | 330+       | ~40           | ~40              |
| String class lookups/<br>frame | ~50        | ~0            | ~0               |
| Creature family<br>factories   | 1 (Mephit) | 1             | 15+              |

Report generated by codebase analysis of `/home/ubuntu/dnd35prototype` . All line numbers reference the codebase as of May 27, 2026.